



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Quizzie - Plataforma multidispositivo para la realización y gestión dinámica de tests educativos

Realizado por
Francisco Gago Vázquez

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Jesús Moreno León

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de julio, curso 2025/26

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: Keyword 1, keyword 2, ..., keyword N

Índice general

1	Introducción	1
1.1.	Contexto y objeto del proyecto	1
1.2.	Motivación	2
1.3.	Objetivos del proyecto	2
1.3.1.	Objetivos académicos	2
1.3.2.	Objetivos técnicos y profesionales	3
1.4.	Resumen del <i>stack</i> tecnológico	4
1.4.1.	Ecosistema del <i>backend</i>	4
1.4.2.	Ecosistema del <i>frontend</i>	5
1.5.	Estructura de la memoria	5
2	Acta de constitución	8
2.1.	Información del proyecto	8
2.2.	Descripción y propósito	8
2.3.	Requerimientos de alto nivel	9
2.4.	Marco legal y normativo	9
2.5.	Riesgos iniciales de alto nivel	10
2.5.1.	Riesgos de alcance	10
2.5.2.	Riesgos técnicos	11
2.5.3.	Riesgos temporales	11
2.6.	Lista de interesados (<i>Stakeholders</i>)	12
3	Gestión del proyecto	13
3.1.	Metodología del trabajo	13
3.2.	Línea base del alcance	14
3.2.1.	Estructura del Desglose del Trabajo (EDT)	14
3.2.2.	Diccionario de la EDT	15
3.3.	Línea base del cronograma	21
3.3.1.	Fases del proyecto	21
3.3.2.	Lista de actividades	22
3.3.3.	Lista de hitos	23
3.3.4.	Diagrama de Gantt	24
3.4.	Línea base de Costes	24
3.4.1.	Análisis de costes	24
3.4.2.	Costes de infraestructura según el número de usuarios	27
3.4.3.	Coste total de propiedad (TCO)	29
3.5.	Plan de gestión de cambios y riesgos	29
4	Estado del arte y fundamentación	31
4.1.	Análisis del mercado	31
4.2.	Factores diferenciadores	32
4.3.	Fundamentación tecnológica	33

4.3.1.	Alternativas consideradas y justificación de las decisiones . . .	33
4.3.2.	Limitaciones tecnológicas	37
4.3.3.	Herramientas de soporte al desarrollo y control de calidad . . .	37
4.4.	Conclusiones del estudio previo	39
5	Análisis de requisitos	40
5.1.	Actores del sistema	40
5.2.	Historias de usuario	40
5.3.	Requisitos de información	40
5.4.	Reglas de negocio y restricciones	40
5.5.	Requisitos funcionales	40
5.5.1.	Criterios de aceptación	40
5.6.	Casos de uso	40
5.7.	Requisitos no funcionales	40
5.8.	Trazabilidad de requisitos	41
6	Diseño y arquitectura	42
6.1.	Patrón arquitectónico elegido	42
6.2.	Estructura modular del código	42
6.3.	Modelo de datos	42
6.4.	Diseño de interfaces	42
6.5.	Seguridad	42
6.6.	Documentación de la API	43
7	Metodología técnica (CI/CD)	44
7.1.	Políticas del repositorio	44
7.2.	Integración y Despliegue Continuo (CI/CD)	44
7.2.1.	Configuración del Pipeline y automatización	44
7.3.	Perfiles de configuración y entornos	44
7.4.	Métricas de calidad y análisis estático de código	44
8	Implementación y pruebas	46
8.1.	Componentes clave	46
8.2.	Estrategia de testing	46
8.3.	Casos de prueba y validación	46
8.4.	Pruebas de carga y rendimiento	46
9	Resultados y evaluación	47
9.1.	Grado de cumplimiento de la Línea Base del Alcance	47
9.2.	Tiempo real frente a Línea Base del Cronograma	47
9.3.	Gastos incurridos frente a Línea Base de Costes	47
9.4.	Desviaciones y gestión de cambios realizados	47
9.5.	Resultados de las pruebas y validación	47
9.6.	Retrospectiva técnica	47
10	Conclusiones	48
10.1.	Grado de cumplimiento de los objetivos	48

10.2. Lecciones aprendidas y valoración personal	48
10.3. Trabajos futuros y líneas de evolución	48
Bibliografía	51

Índice de figuras

3.1. Estructura de Desglose del Trabajo (EDT).	15
3.2. Diagrama de Gantt inicial del proyecto.	24
3.3. Costes mensuales de infraestructura según escenario de escalado. . .	28
3.4. Evolución de OPEX en función del volumen de usuarios.	28

Índice de tablas

2.1. Datos de identificación del proyecto.	8
2.2. Tabla de riesgos de alcance.	10
2.3. Tabla de riesgos técnicos.	11
2.4. Tabla de riesgos temporales.	11
2.5. Lista de interesados y roles en el proyecto.	12
3.1. Datos del entregable 1.1.	15
3.2. Actividades del entregable 1.1.	15
3.3. Datos del entregable 1.2.	16
3.4. Actividades del entregable 1.2.	16
3.5. Datos del entregable 1.3.	16
3.6. Actividades del entregable 1.3.	16
3.7. Datos del entregable 2.1.	17
3.8. Actividades del entregable 2.1.	17
3.9. Datos del entregable 2.2.	17
3.10. Actividades del entregable 2.2.	17
3.11. Datos del entregable 3.1.	18
3.12. Actividades del entregable 3.1.	18
3.13. Datos del entregable 3.2.	18
3.14. Actividades del entregable 3.2.	18
3.15. Datos del entregable 3.3.	19
3.16. Actividades del entregable 3.3.	19
3.17. Datos del entregable 4.1.	19
3.18. Actividades del entregable 4.1.	19
3.19. Datos del entregable 4.2.	20
3.20. Actividades del entregable 4.2.	20
3.21. Datos del entregable 4.3.	20
3.22. Actividades del entregable 4.3.	20
3.23. Fases temporales del proyecto.	21
3.24. Actividades organizadas por su fase del cronograma.	22
3.25. Hitos de control del proyecto.	23
3.26. Resumen de Gastos de Capital (CAPEX).	25
3.27. Costes netos laborales agrupados por perfil profesional.	26
3.28. Resumen de Gastos Operativos mensuales (OPEX).	27
3.29. Comparativa de OPEX según escenario de escalado.	27
3.30. TCO proyectado a tres años.	29
4.1. Tabla comparativa del mercado.	32
4.2. Resumen del stack tecnológico de Quizzie.	33
4.3. Alternativas tecnológicas para la base de datos.	34
4.4. Alternativas tecnológicas para el ORM y validación.	34
4.5. Alternativas tecnológicas para el desarrollo del backend.	35
4.6. Alternativas para el lenguaje de programación frontend.	35

4.7. Alternativas tecnológicas para la interfaz de usuario.	36
4.8. Alternativas tecnológicas para el diseño de estilos visuales.	36
4.9. Alternativas para la infraestructura y el despliegue.	37

Índice de extractos de código

1. Introducción

A lo largo de mi trayectoria en la carrera, he asistido a multitud de clases teóricas y prácticas, y si algo he aprendido de ellas es que mantener un dinamismo activo en el aula marca la diferencia en el proceso de aprendizaje. Esta experiencia de estudiante, sumada al reto de diseñar un producto tecnológico completo y autónomo, me motivó a enfocar mi Trabajo de Fin de Grado en el desarrollo de *Quizzie*, una plataforma orientada a transformar las dinámicas de evaluación en el entorno educativo.

El objetivo de este capítulo inicial es asentar las bases del trabajo y ofrecer una hoja de ruta clara. Para ello, en las siguientes secciones se detalla el contexto y objeto del proyecto, las motivaciones personales y profesionales que impulsaron su creación, así como los objetivos académicos y técnicos que se pretenden alcanzar.

1.1. Contexto y objeto del proyecto

A raíz de la creciente digitalización en la enseñanza y la necesidad de mantener el interés de un alumnado, surge la necesidad de buscar soluciones que adapten la forma de evaluar en las aulas a la realidad de los estudiantes. En la primera reunión con el tutor de este proyecto, planteamos el reto de diseñar una herramienta que facilitara el trabajo de los profesores y que, al mismo tiempo, resultara entretenida y motivadora para los alumnos a la hora de comprobar lo aprendido mediante cuestionarios en clase.

La integración de este tipo de cuestionarios rápidos ofrece grandes ventajas en el día a día del aula. Para el alumno, rompe la monotonía de las clases teóricas convencionales introduciendo **aprendizaje dinámico**, elevando la participación y proporcionando *feedback* instantáneo sobre su nivel de comprensión. Por otro lado, para el profesor funciona como una herramienta de **seguimiento en tiempo real**: le permite medir de manera automatizada si el grupo está asimilando los conceptos explicados y, en caso de detectar fallos comunes, aclarar las dudas en ese mismo momento.

Sin embargo, el uso de las plataformas actuales en entornos educativos reales destapa un **problema de control crítico**. Al basarse generalmente en la difusión de un enlace o un código de acceso, es muy sencillo que los estudiantes compartan estas credenciales por grupos de mensajería. Esto provoca que alumnos que no han asistido a clase, o que se encuentran en su casa, puedan responder al cuestionario a distancia. Esta falta de **verificación de la presencia física** falsea las estadísticas de rendimiento que recibe el docente e impide usar estos test como herramientas de **evaluación formal**.

Como respuesta a este escenario, el objeto de este proyecto es el desarrollo de *Quizzie*. La aplicación está diseñada para mantener la sencillez y el dinamismo que

se busca en los cuestionarios en vivo, pero introduciendo una **capa de verificación** para los alumnos. A través de este mecanismo de control, *Quizzie* asegura que la participación se acote estrictamente a los alumnos presentes en el aula, evitando el **fraude a distancia** y consolidándose como un instrumento de evaluación válido y seguro.

1.2. Motivación

La elección de este proyecto nace, en gran medida, de mi propia experiencia como estudiante a lo largo de mi formación. He vivido en primera persona cómo la introducción de **cuestionarios interactivos** en mitad de una clase es capaz de transformar por completo el clima del aula, despertando el interés de los alumnos y fijando los conceptos de una manera mucho más eficaz que los métodos tradicionales. No obstante, también he sido testigo de las limitaciones de las herramientas actuales: el uso de pseudónimos informales, la posibilidad de participar de forma remota sin acudir a clase o la frustración del docente al no poder **validar la autenticidad** de los resultados. Ver cómo una dinámica con tanto potencial perdía rigor por falta de control o por limitaciones de las plataformas utilizadas fue el detonante que me impulsó a buscar esta solución.

Desde un punto de vista profesional, mi motivación principal se centra en el reto de plasmar de forma integral todos los conocimientos y competencias adquiridos a lo largo de la carrera. El desarrollo de *Quizzie* representa la oportunidad perfecta para demostrar mi capacidad de afrontar un **proyecto de software completo**, responsabilizándome de cada una de sus fases: desde las etapas iniciales de concepción, estudio previo y análisis de requisitos, hasta el diseño arquitectónico, la implementación técnica y la validación mediante pruebas. Abordar este **ciclo de vida de extremo a extremo** no solo me permite consolidar mis conocimientos técnicos, sino también validar mi criterio al tomar decisiones justificadas ante problemas, convirtiendo este trabajo en el puente definitivo hacia mi madurez como ingeniero de *software*.

1.3. Objetivos del proyecto

Para guiar el desarrollo del proyecto y asegurar el cumplimiento de las expectativas tanto formativas como técnicas, he definido una serie de metas divididas en dos vertientes: académica y técnico-profesional.

1.3.1. Objetivos académicos

Metas orientadas a consolidar la formación recibida durante los estudios de grado:

- **Integrar y aplicar** de forma conjunta los conocimientos teóricos y prácticos adquiridos en las distintas asignaturas de la carrera.
- **Adoptar metodologías de trabajo ágiles** y estándares de la industria en la gestión y planificación temporal del proyecto.
- **Fomentar el uso de buenas prácticas** de ingeniería del *software*, incluyendo el control de versiones formal, el diseño de código limpio y la documentación sistemática.

1.3.2. Objetivos técnicos y profesionales

Retos tecnológicos asumidos para el producto final y el desarrollo de competencias laborales:

- **Dominar nuevas tecnologías** y herramientas del ecosistema de desarrollo actual, tales como el *framework* FastAPI para el entorno de servidor y React para la interfaz de usuario.
- **Gestionar de manera autónoma el ciclo de vida completo** de un producto de *software* real, abarcando desde su concepción y análisis hasta su despliegue y evaluación.
- **Diseñar e implementar un sistema de verificación** robusto y eficiente que resuelva de manera efectiva la problemática de la asistencia y autoría en las evaluaciones en el aula.
- **Asegurar la calidad del *software***: Diseñar e implementar un plan de pruebas automatizadas combinado con herramientas de análisis estático de código, garantizando la mantenibilidad, seguridad y fiabilidad.
- **Implementar una estrategia de pruebas exhaustiva** que abarque *tests* unitarios y de integración, asegurando la fiabilidad del código y el correcto funcionamiento de las reglas de negocio del sistema.
- **Diseñar y configurar un flujo de trabajo basado en CI/CD** (Integración y Despliegue Continuo) mediante herramientas de automatización, garantizando que cada cambio en el repositorio se verifique y despliegue de manera segura y eficiente.

Objetivos de la plataforma

Aspiraciones funcionales y operativas de *Quizzie*:

- **Evaluación docente simplificada**: Proveer al profesorado de un sistema eficiente para gestionar cuestionarios que sirvan de herramienta de evaluación.
- **Dinamismo e interactividad en el aula**: Priorizar la fluidez de las sesiones en tiempo real mediante mecánicas gamificadas que fomenten una participación activa y competitiva por parte de los estudiantes.

- **Seguimiento continuo del alumnado:** Ofrecer herramientas analíticas que permitan monitorizar el rendimiento individual y colectivo, ayudando a identificar las necesidades de aprendizaje durante el curso.
- **Validación y presencialidad física:** Asegurar la legitimidad de las pruebas mediante mecanismos de verificación digital *in situ*, garantizando que los resultados registrados correspondan exclusivamente a los alumnos presentes en el aula.
- **Accesibilidad y agilidad de incorporación:** Optimizar la experiencia de usuario con flujos de acceso inmediato y sin necesidad de registro previo para los estudiantes, facilitando una entrada instantánea a las salas de cuestionarios.
- **Compatibilidad y despliegue multidispositivo:** Garantizar que la plataforma sea completamente responsiva y accesible desde cualquier tipo de dispositivo (móviles, *tablets* u ordenadores), independientemente de su sistema operativo o navegador.
- **Garantía de privacidad y cumplimiento:** Blindar el almacenamiento y tratamiento de los datos académicos bajo el estricto cumplimiento del marco legal y de protección de datos vigente.

1.4. Resumen del *stack* tecnológico

Para garantizar el cumplimiento de los objetivos técnicos del proyecto y asegurar un desarrollo robusto, eficiente y escalable, se ha seleccionado un conjunto de herramientas alineadas con los estándares de la industria actual. Esta sección ofrece una visión inicial del *stack* tecnológico empleado para la construcción de *Quizzie*. Las ventajas técnicas de cada opción frente a otras alternativas del mercado se argumentarán en profundidad en el Capítulo 4.

El núcleo del sistema se divide claramente bajo una arquitectura cliente-servidor, donde el *backend* gestiona la lógica de negocio, la seguridad y la comunicación en tiempo real, mientras que el *frontend* ofrece una experiencia interactiva y fluida para profesores y alumnos.

1.4.1. Ecosistema del *backend*

El entorno se ha diseñado priorizando el alto rendimiento, la seguridad y la agilidad en el desarrollo. Las tecnologías y herramientas principales que componen esta capa son:

- **Lenguaje de programación:** Python 3.12, aprovechando las últimas mejoras de rendimiento y tipado estático del lenguaje.
- **Framework de desarrollo:** FastAPI, seleccionado por su alta velocidad de ejecución, soporte nativo de operaciones asíncronas y validación automática

de datos.

- **Comunicación en tiempo real:** Protocolo WebSocket, implementado para gestionar de manera bidireccional y eficiente el flujo de eventos y la sincronización de las salas en vivo.
- **Persistencia y ORM:** SQLAlchemy en combinación con SQLite. Al no requerir un gestor de base de datos externo complejo en esta fase, esta solución permite interactuar de forma nativa con la base de datos utilizando objetos de Python de manera eficiente.
- **Gestión de dependencias:** uv, utilizado como gestor de paquetes ultrarrápido para optimizar los tiempos de instalación y asegurar la reproducibilidad del entorno.

1.4.2. Ecosistema del *frontend*

La interfaz de usuario se ha planteado como una *Single Page Application* (SPA) responsiva para ofrecer la fluidez que requiere una aplicación con dinámicas lúdicas y en vivo:

- **Framework principal:** React, para la construcción de una interfaz basada en componentes reutilizables y reactivos.
- **Herramienta de construcción (Build Tool):** Vite, que proporciona un entorno de desarrollo ágil gracias a su velocidad de compilación y recarga en caliente.
- **Lenguajes:** TypeScript como eje principal para dotar al código de tipado fuerte y robustez, apoyado puntualmente en JavaScript.
- **Enrutado y consumo de API:** React Router Dom para la navegación interna de la aplicación y Axios para realizar peticiones asíncronas y comunicarse con el *backend*.

1.5. Estructura de la memoria

Para facilitar la lectura y comprensión de este documento, la memoria se ha estructurado en diez capítulos principales y dos manuales adicionales organizados de la siguiente manera:

- **Capítulo 1: Introducción.** Define el punto de partida del proyecto, contextualizando el problema de la evaluación presencial frente al uso de herramientas digitales. Se exponen la motivación, los objetivos del proyecto y una visión general del *stack* tecnológico seleccionado.
- **Capítulo 2: Acta de constitución.** Formaliza el inicio del proyecto, detallando la descripción general del producto, los requerimientos de alto nivel, el marco legal y normativo (incluyendo cumplimiento de RGPD y licencias), los riesgos preliminares y los interesados o *stakeholders*.

- **Capítulo 3: Gestión del proyecto.** Describe la planificación organizativa inicial, incluyendo la metodología de trabajo, la línea base del alcance (a través de la EDT y su diccionario), la línea base del cronograma (con el diagrama de Gantt e hitos), la línea base de costes (estimación de presupuestos, Capex, Opex y TCO), así como los mecanismos para la gestión de riesgos y solicitudes de cambios.
- **Capítulo 4: Estado del arte y fundamentación.** Presenta un análisis comparativo de las soluciones y competidores existentes en el mercado de la gamificación educativa, identificando los factores diferenciales de *Quizzie*. Además, justifica detalladamente las decisiones tecnológicas del *stack* y las alternativas consideradas.
- **Capítulo 5: Análisis de requisitos.** Modela los requisitos del sistema empleando casos de uso, historias de usuario y reglas de negocio. Especifica tanto los requisitos funcionales como los no funcionales, y establece la matriz de trazabilidad que vincula los objetivos iniciales con las pruebas del sistema.
- **Capítulo 6: Diseño y arquitectura.** Detalla la arquitectura de *software* seleccionada (cliente-servidor estructurada en capas y basada en eventos), la modularización del código en el *frontend* y el *backend*, el modelo de datos relacional mediante diagramas de dominio, el diseño de la interfaz de usuario (*mockups* iniciales frente al acabado final), los mecanismos de seguridad (JWT y *tokens* de acceso) y la documentación de la API con OpenAPI/Swagger.
- **Capítulo 7: Metodología técnica (CI/CD).** Detalla los aspectos de ingeniería y DevOps integrados en el repositorio, abarcando las políticas de ramas y *commits* (Commitizen, *hooks* de *pre-commit*), el *pipeline* de integración y despliegue continuo (GitHub Actions), la gestión de perfiles de configuración y secretos, y el control de calidad estática mediante SonarQube.
- **Capítulo 8: Implementación y pruebas.** Recoge las decisiones del desarrollo real mostrando fragmentos de código significativos. Describe la estrategia de *testing* (pruebas unitarias, de integración y extremo a extremo), los umbrales de cobertura y el diseño de las pruebas de carga y rendimiento de la plataforma.
- **Capítulo 9: Resultados y evaluación.** Expone el análisis tras la finalización del desarrollo, comparando los datos de ejecución reales con la planificación inicial (desviaciones en alcance, tiempos y costes). Asimismo, presenta los resultados de los casos de prueba ejecutados y una retrospectiva técnica enfocada en la **deuda técnica acumulada**.
- **Capítulo 10: Conclusiones.** Evalúa el **grado de consecución de los objetivos** generales e individuales planteados. Recoge las lecciones aprendidas durante el ciclo de vida del proyecto, una valoración personal sobre el proceso y las futuras líneas de evolución de la plataforma.

Adicionalmente, se incluyen dos manuales prácticos al final del documento como anexos independientes:

- **Manual de instalación:** Guía técnica detallada para el despliegue del sistema

tanto en un entorno local de desarrollo como en entornos de producción en la nube.

- **Manual de usuario:** Guía visual e instructiva orientada a profesores y alumnos para el correcto uso y manejo de la plataforma.

2. Acta de constitución

El desarrollo de *Quizzie* requiere fijar un marco formal que defina sus reglas y compromisos iniciales antes de comenzar cualquier fase de diseño o desarrollo técnico. El propósito de este capítulo es establecer la línea base del proyecto, delimitando su alcance inicial, sus interesados y las restricciones bajo las que se ejecutará. Este documento define el propósito estratégico de la aplicación, garantizando desde el primer momento la viabilidad técnica, la gestión de riesgos y el estricto cumplimiento del marco legal establecido.

2.1. Información del proyecto

La formalización de *Quizzie* como Trabajo de Fin de Grado requiere establecer, en primer lugar, los datos de identificación administrativa y académica que muestran su autoría y tutorización.

Nombre del proyecto	Quizzie
Descripción	Plataforma multidispositivo para la realización y gestión dinámica de tests educativos
Tipo de proyecto	Trabajo de Fin de Grado (TFG)
Titulación	Grado en Ingeniería Informática - Ingeniería del Software
Autor	Francisco Gago Vázquez
Tutor	Jesús Moreno León
Departamento	Lenguajes y Sistemas Informáticos (LSI)

Tabla 2.1: Datos de identificación del proyecto.

2.2. Descripción y propósito

Quizzie nace como una plataforma multidispositivo orientada a transformar la dinámica de evaluación en el entorno educativo. El propósito fundamental de su desarrollo es ofrecer una herramienta interactiva que agilice la creación, gestión y ejecución de tests en tiempo real. Frente a soluciones comerciales existentes, la aplicación pone el foco en la inmediatez, la robustez técnica y el control del flujo de evaluación por parte del profesor gracias a la verificación de la presencialidad de

los alumnos. El desglose de estas capacidades se materializará detalladamente en el capítulo de [análisis de requisitos](#).

2.3. Requerimientos de alto nivel

El éxito de la plataforma depende del cumplimiento de una serie de condiciones críticas, atributos de calidad y restricciones de diseño iniciales. Estas directrices sientan las bases del desarrollo y actúan como los criterios de aceptación esenciales para considerar el producto como terminado y listo para su lanzamiento.

Para validar el correcto estado del sistema, este debe cumplir estrictamente con la siguiente lista de verificación:

- **Disponibilidad y despliegue en la nube:** La plataforma completa debe estar desplegada en un entorno de producción real y accesible a través de internet para sus usuarios.
- **Compatibilidad multiplataforma:** La interfaz debe ser completamente *responsive*, garantizando que los alumnos interactúen de forma fluida desde dispositivos móviles y el profesor desde equipos de escritorio.
- **Simplicidad de uso:** El diseño de las pantallas debe priorizar la usabilidad, permitiendo el acceso a las salas de cuestionarios y la creación de test con el menor número de interacciones posible.
- **Seguridad en las comunicaciones:** El intercambio de datos en tiempo real debe realizarse bajo protocolos seguros (HTTPS y WSS), asegurando el cifrado de la información en tránsito.
- **Privacidad de los datos:** El almacenamiento del histórico de calificaciones y datos académicos debe cumplir estrictamente con el marco legal de protección de datos vigente.

2.4. Marco legal y normativo

El desarrollo y uso de *Quizzie* se rige por la normativa vigente en materia de protección de datos, seguridad de la información y propiedad intelectual. Al tratarse de una herramienta orientada al entorno educativo, el diseño del sistema ha priorizado el principio de minimización de datos para mitigar los riesgos asociados al tratamiento de información sensible.

A continuación, se detallan los pilares normativos y legales de la aplicación:

- **Reglamento General de Protección de Datos (RGPD):** La plataforma aplica el principio de privacidad desde el diseño. Para los estudiantes, el acceso a las salas se realiza mediante un código PIN y su *uvvus* (*Usuario Virtual de la Universidad de Sevilla*), prescindiendo de un registro tradicional y limitando el almacenamiento de datos personales a este identificador institucional único.

En el caso de los docentes, el registro obligatorio se limita a un correo electrónico y un alias, omitiendo identificadores directos como nombres, apellidos o DNI.

- **Seguridad y persistencia de calificaciones:** El histórico de resultados almacenado en el sistema solo vincula las calificaciones al *uvus* del alumno y cuenta con la previa verificación física del profesor. Esto garantiza la integridad del proceso de evaluación sin exponer la identidad real del estudiantado en la base de datos.
- **Propiedad intelectual y licenciamiento:** El código fuente y los entregables de este Trabajo de Fin de Grado se distribuyen bajo una licencia de código abierto MIT. Esta elección garantiza la total transparencia del desarrollo, permite la auditoría pública del sistema y fomenta su reutilización o extensión por parte de la comunidad académica, eximiendo al autor de cualquier responsabilidad derivada de su uso.

2.5. Riesgos iniciales de alto nivel

El desarrollo de una plataforma orientada a la evaluación en tiempo real conlleva una serie de riesgos de alcance, técnicos y temporales. Identificarlos de forma temprana permite establecer el impacto potencial de cada amenaza y diseñar estrategias de mitigación que garanticen el éxito del proyecto y el cumplimiento del cronograma.

A continuación, se detallan los riesgos iniciales organizados por su naturaleza y evaluados según su probabilidad de ocurrencia y el impacto estimado en el desarrollo global del proyecto.

2.5.1. Riesgos de alcance

Desviaciones relacionadas con el volumen de funcionalidades implementadas y la correcta definición de los requisitos del sistema:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-01	Corrupción del alcance (<i>Scope Creep</i>) al querer añadir más modos de juego o analíticas complejas.	Definición estricta del Producto Mínimo Viable (MVP). Las mejoras secundarias se derivan a "Trabajos Futuros".	Medio	Alto
R-02	Ambigüedad en los requisitos iniciales que obligue a rehacer lógica de negocio ya programada.	Validación previa del modelo de datos e historias de usuario mediante diagramas de flujo antes de codificar.	Bajo	Medio

Tabla 2.2: Tabla de riesgos de alcance.

2.5.2. Riesgos técnicos

Incidencias tecnológicas ligadas a la arquitectura del sistema, conectividad de red e infraestructura empleada:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-03	Saturación del servidor o retardos (<i>lag</i>) ante múltiples alumnos conectados a la vez.	Diseñar un modelo de datos ligero para las tramas de WebSockets y realizar pruebas de carga tempranas.	Medio	Alto
R-04	Indisponibilidad, caídas o latencia elevada en los servicios en la nube de terceros (<i>Gemini</i> o <i>Render</i>).	Implementación de zonas de control de errores (<i>error boundaries</i>) y reintentos exponenciales en el cliente.	Bajo	Alto
R-05	Despliegue y red: fallos de conectividad o puertos bloqueados en la red wifi que impidan la comunicación.	Configurar el despliegue bajo protocolos seguros de producción y prever mecanismos de reconexión rápida.	Bajo	Alto
R-06	Incompatibilidades visuales menores o descuadres de la interfaz en ciertos navegadores o pantallas antiguas.	Uso de CSS responsivo estándar mediante <i>Flexbox/Grid</i> y testeo rápido en Google Chrome y Firefox.	Alto	Bajo

Tabla 2.3: Tabla de riesgos técnicos.

2.5.3. Riesgos temporales

Amenazas asociadas directa o indirectamente con el cumplimiento de los plazos y el calendario de entregas de la memoria y el software:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-07	Retraso en el calendario por la curva de aprendizaje en WebSockets o la API de <i>Gemini Pro</i> .	Reservar las primeras semanas para crear pruebas de concepto aisladas antes de programar el núcleo.	Medio	Alto
R-08	Falta de disponibilidad por la carga de trabajo de otras asignaturas simultáneas del curso.	Planificación mediante <i>sprints</i> semanales rígidos con objetivos mínimos viables para mantener el ritmo.	Alto	Medio
R-09	Retrasos menores en la redacción de la memoria final debido al foco exclusivo en el desarrollo de código.	Establecer bloques de tiempo semanales dedicados a documentar los módulos que se acaban de programar.	Medio	Bajo

Tabla 2.4: Tabla de riesgos temporales.

2.6. Lista de interesados (*Stakeholders*)

En este apartado se identifica a los actores, grupos e instituciones que interactúan con la plataforma o se ven afectados por su desarrollo, definiendo sus roles y expectativas principales.

Interesado	Rol en el proyecto	Intereses / expectativas
Autor	Desarrollar e implementar el sistema completo.	Diseñar una arquitectura robusta, cumplir los plazos de entrega y superar con éxito la defensa del TFG.
Tutor	Supervisar, guiar y evaluar el desarrollo del proyecto.	Garantizar el rigor metodológico, la calidad técnica del software y validar el cumplimiento de los objetivos.
Departamento LSI	Entidad académica de la Universidad responsable de la calificación del TFG.	Asegurar el cumplimiento de la normativa de los Trabajos de Fin de Grado y el correcto proceso de evaluación.
Cuerpo docente	Futuros usuarios (Profesores).	Disponer de una herramienta intuitiva que facilite la evaluación de los alumnos y asegure su presencialidad.
Alumnado	Futuros usuarios (Estudiantes).	Acceder de forma ágil y multidispositivo sin registros complejos para participar en las evaluaciones dinámicas.

Tabla 2.5: Lista de interesados y roles en el proyecto.

3. Gestión del proyecto

En la ingeniería de software, el éxito de un proyecto no depende únicamente de la calidad de su código, sino de una rigurosa gobernanza que garantice la viabilidad técnica, temporal y económica del proyecto. Para ello, se adopta un enfoque predictivo en la definición de las líneas base, complementado con prácticas ágiles para la gestión y ejecución del cambio.

A lo largo de las siguientes secciones se detalla la metodología que se seguirá a lo largo del desarrollo, se establecen formalmente las línea base del alcance, cronograma y de costes, y se presenta el plan de gestión de riesgos y cambios destinado a mitigar las desviaciones del proyecto.

3.1. Metodología del trabajo

El desarrollo de *Quizzie* se organiza gracias a un flujo de trabajo ágil centralizado en GitHub. Para el día a día y el seguimiento del proyecto, se utilizan las siguientes herramientas:

- **GitHub:** Centraliza el almacenamiento del código fuente y la gestión de las tareas mediante las siguientes herramientas:
 - **Issues y Milestones:** Cada funcionalidad o corrección se registra como una *Issue* (tarea). Estas se agrupan en *Milestones* (hitos temporales) para asociar el avance del código con los plazos asignados en el cronograma.
 - **Ramas permanentes:** Se mantiene una separación estricta entre la rama *main* (exclusiva para versiones definitivas y estables) y la rama *develop* (entorno de integración donde se vuelcan las novedades).
 - **Ramas de tareas:** Para cada *issue* se abre una rama específica desde *develop* (ej. *feature/websocket-timer*). Una vez completada y probada la tarea de forma aislada, esta rama se fusiona (*merge*) de vuelta en *develop*.
 - **Releases:** Al completar un *Milestone* y consolidar los cambios en *main*, se genera una *Release* con su etiqueta de versión.
- **Clockify:** Se utiliza exclusivamente para cronometrar el tiempo real dedicado a cada tarea e hito, permitiendo controlar el esfuerzo invertido.
- **Outlook:** Canal formal para comunicaciones con el tutor y envío de entregables o documentación.

El avance del proyecto se valida de forma continua mediante tutorías. La dinámica de estas reuniones consiste en presentar y revisar el software funcionando o el texto redactado hasta esa fecha para recibir el visto bueno del

tutor y, acto seguido, definir los siguientes pasos y prioridades a realizar para la próxima tutoría.

3.2. Línea base del alcance

La línea base del alcance define con precisión los límites del proyecto, asegurando que todos los esfuerzos se concentren exclusivamente en los entregables necesarios para cumplir con los objetivos académicos y técnicos. A continuación, se detallan los supuestos, restricciones y exclusiones que condicionan esta planificación.

- **Supuestos:** Se asume la disponibilidad continua de las infraestructuras en la nube (proveedores de alojamiento o bases de datos) durante las fases de desarrollo y despliegue, así como el acceso ilimitado a la documentación de las API y tecnologías que componen el *stack* tecnológico.
- **Restricciones:** El proyecto está sujeto a una restricción temporal vinculada a los plazos institucionales de entrega y defensa del Trabajo Fin de Grado. Asimismo, el desarrollo debe ceñirse al presupuesto simulado y a la capacidad operativa de un único desarrollador.
- **Exclusiones:** Queda fuera del alcance del proyecto el despliegue comercial en producción y el soporte para navegadores web obsoletos o sin compatibilidad con WebSockets modernos.

3.2.1. Estructura del Desglose del Trabajo (EDT)

Para organizar y estructurar el alcance total del proyecto, se ha elaborado la siguiente Estructura de Desglose del Trabajo (EDT). Esta representación jerárquica descompone el proyecto en paquetes de trabajo, divididos a su vez en entregables específicos que se detallarán en el diccionario de la EDT ([3.2.2](#)).

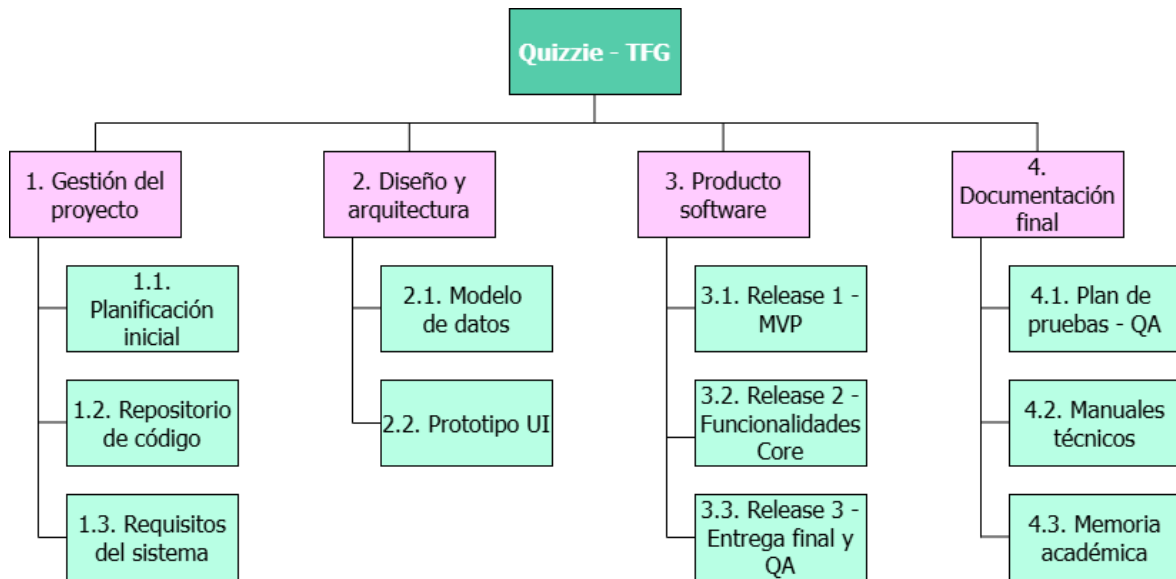


Figura 3.1: Estructura de Desglose del Trabajo (EDT).

3.2.2. Diccionario de la EDT

El diccionario de la EDT define formalmente el alcance de cada entregable y desglosa las actividades necesarias para su consecución, especificando los perfiles profesionales asignados y la estimación temporal.

Paquete 1.0: Gestión del proyecto

ID Entregable	1.1
Nombre	Planificación inicial
Descripción	Acta de constitución del proyecto y cronograma base estimado para fijar los hitos académicos.
Paquete	1. Gestión del proyecto

Tabla 3.1: Datos del entregable 1.1.

Actividad	Recurso	Horas
ACT-01	Project Manager	4h
ACT-02	Project Manager	4h

Tabla 3.2: Actividades del entregable 1.1.

ID Entregable	1.2
Nombre	Repositorio de código
Descripción	Configuración de GitHub con reglas de protección de ramas, flujos de trabajo basados en fusiones (<i>merges</i>) y plantillas estandarizadas para <i>issues</i> .
Paquete	1. Gestión del proyecto

Tabla 3.3: Datos del entregable 1.2.

Actividad	Recurso	Horas
ACT-13	Técnico	2h
ACT-14	Project Manager	5h
ACT-15	Project Manager	3h

Tabla 3.4: Actividades del entregable 1.2.

ID Entregable	1.3
Nombre	Requisitos del sistema
Descripción	Catálogo de requisitos funcionales y no funcionales, junto al modelado de casos de uso y flujos <i>core</i> de la aplicación.
Paquete	1. Gestión del proyecto

Tabla 3.5: Datos del entregable 1.3.

Actividad	Recurso	Horas
ACT-08	Analista	3h
ACT-09	Analista	4h

Tabla 3.6: Actividades del entregable 1.3.

Paquete 2.0: Diseño y Arquitectura

ID Entregable	2.1
Nombre	Modelo de datos
Descripción	Diseño conceptual, lógico y físico de la base de datos relacional, materializado en el diagrama de entidades del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.7: Datos del entregable 2.1.

Actividad	Recurso	Horas
ACT-10	Analista	4h
ACT-11	Analista	4h
ACT-12	Programador Backend	2h

Tabla 3.8: Actividades del entregable 2.1.

ID Entregable	2.2
Nombre	Prototipo UI
Descripción	Diseños visuales y plantillas para las pantallas del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.9: Datos del entregable 2.2.

Actividad	Recurso	Horas
ACT-03	Diseñador UI	2h
ACT-04	Diseñador UI	5h
ACT-05	Diseñador UI	5h

Tabla 3.10: Actividades del entregable 2.2.

Paquete 3.0: Producto software

ID Entregable	3.1
Nombre	Release 1 (MVP)
Descripción	Habilitación y despliegue del flujo completo base del sistema, permitiendo la creación de cuestionarios por parte del profesor y la realización de los mismos por los alumnos.
Paquete	3. Producto software

Tabla 3.11: Datos del entregable 3.1.

Actividad	Recurso	Horas
ACT-16	Programador Backend	25h
ACT-17	Programador Frontend	35h
ACT-18	Programador Backend	20h

Tabla 3.12: Actividades del entregable 3.1.

ID Entregable	3.2
Nombre	Release 2 (Funcionalidades Core)
Descripción	Implementación de la capa de comunicación en tiempo real mediante WebSockets y el motor de validación de datos del sistema.
Paquete	3. Producto software

Tabla 3.13: Datos del entregable 3.2.

Actividad	Recurso	Horas
ACT-21	Programador Backend	20h
ACT-22	Programador Frontend	20h
ACT-23	Programador Backend	12h

Tabla 3.14: Actividades del entregable 3.2.

ID Entregable	3.3
Nombre	Release 3 (Entrega final y QA)
Descripción	Integración de funcionalidades avanzadas de gestión, pulido general de la interfaz y despliegue del sistema.
Paquete	3. Producto software

Tabla 3.15: Datos del entregable 3.3.

Actividad	Recurso	Horas
ACT-24	Programador Backend	5h
ACT-25	Programador Frontend	5h
ACT-26	Técnico	3h

Tabla 3.16: Actividades del entregable 3.3.

Paquete 4.0: Documentación final

ID Entregable	4.1
Nombre	Plan de pruebas (QA)
Descripción	Diseño y ejecución de baterías de test de integración y <i>scripts</i> específicos de pruebas de carga.
Paquete	4. Documentación final

Tabla 3.17: Datos del entregable 4.1.

Actividad	Recurso	Horas
ACT-19	Tester	7h
ACT-20	Tester	8h

Tabla 3.18: Actividades del entregable 4.1.

ID Entregable	4.2
Nombre	Manuales técnicos
Descripción	Redacción del manual de usuario y del manual de despliegue.
Paquete	4. Documentación final

Tabla 3.19: Datos del entregable 4.2.

Actividad	Recurso	Horas
ACT-06	Analista	2h
ACT-07	Técnico	1h

Tabla 3.20: Actividades del entregable 4.2.

ID Entregable	4.3
Nombre	Memoria académica
Descripción	Redacción técnica, maquetación y consolidación del documento final del TFG junto con el diseño de las diapositivas para la defensa.
Paquete	4. Documentación final

Tabla 3.21: Datos del entregable 4.3.

Actividad	Recurso	Horas
ACT-27	Analista	70h
ACT-28	Diseñador UI	10h
ACT-29	Project Manager	10h

Tabla 3.22: Actividades del entregable 4.3.

3.3. Línea base del cronograma

A partir de los paquetes de trabajo presentes en la Estructura de Desglose del Trabajo (EDT), se ha planificado la distribución temporal del proyecto. El flujo de trabajo se estructura dividiendo el ciclo de vida en fases temporales que agrupan las actividades y culminan en hitos de control.

3.3.1. Fases del proyecto

El proyecto se divide en 10 fases temporales consecutivas y paralelas, que se han organizado en 4 bloques de trabajo fundamentales:

ID	Nombre	Descripción	Bloque
F-01	Planificación	Planificación inicial, estimación de tiempos y recursos.	Gestión y planificación
F-02	Documentación	Redacción de manuales y documentación técnica de soporte.	Documentación y cierre
F-03	Inicio	Configuración del entorno base y análisis de requisitos.	Gestión y planificación
F-04	CI/CD	Automatización de flujos de trabajo e integración continua.	Ciclo de vida y calidad
F-05	MVP	Construcción de la primera versión mínima funcional.	Desarrollo de código
F-06	Testing	Diseño y ejecución de la batería de pruebas del sistema.	Ciclo de vida y calidad
F-07	Core	Desarrollo de las características principales (WebSockets).	Desarrollo de código
F-08	QA	Estabilización de código y corrección de errores finales.	Desarrollo de código
F-09	Memoria	Consolidación, revisión final y entrega de la memoria.	Documentación y cierre
F-10	Defensa	Preparación del material de soporte y ensayos.	Documentación y cierre

Tabla 3.23: Fases temporales del proyecto.

3.3.2. Lista de actividades

Cada una de las actividades del cronograma conserva una relación directa con los paquetes de trabajo definidos en el diccionario de la EDT. Para garantizar el seguimiento y la trazabilidad temporal, a continuación se detallan todas las tareas del proyecto:

Fase	ID	Actividad
F-01 – Planificación	ACT-01	Redacción del acta de constitución
	ACT-02	Definición de hitos y estimación del cronograma inicial
	ACT-03	Elección de estilos, paleta de colores y componentes básicos
	ACT-04	Diseño de prototipos de las pantallas de gestión del profesor
	ACT-05	Diseño de la interfaz de respuesta para pantallas de alumnos
F-02 – Documentación	ACT-06	Elaboración del manual funcional como guía para usuarios
	ACT-07	Redacción del manual de despliegue técnico de infraestructura
	ACT-08	Elicitación y redacción de requisitos funcionales y no funcionales
	ACT-09	Modelado de diagramas de casos de uso y flujos esenciales
	ACT-10	Diseño del modelo conceptual y lógico de datos
	ACT-11	Modelado del diagrama de entidades relacional
	ACT-12	Generación del <i>script</i> de inicialización de tablas y poblado
F-03 – Inicio	ACT-13	Inicialización del repositorio en GitHub y estructura base
F-04 – CI/CD	ACT-14	Configuración de reglas en ramas y automatización base
	ACT-15	Diseño y despliegue de <i>templates</i> para <i>issues</i>
F-05 – MVP	ACT-16	Diseño y desarrollo de la persistencia base para cuestionarios
	ACT-17	Implementación de la interfaz de creación de tests y panel base
	ACT-18	Lógica de control y validación para la ejecución asíncrona
F-06 – Testing	ACT-19	Desarrollo y ejecución de test de integración de servicios
	ACT-20	Programación y lanzamiento de <i>scripts</i> de carga
F-07 – Core	ACT-21	Desarrollo de la infraestructura síncrona y canales WebSocket
	ACT-22	Implementación en frontend de la conexión reactiva a salas
	ACT-23	Programación del motor de validación y control de integridad
F-08 – QA	ACT-24	Implementación del módulo de autenticación y panel avanzado
	ACT-25	Desarrollo de vistas de historial y exportación de datos
	ACT-26	Puesta a punto, empaquetado, corrección de errores y despliegue
F-09 – Memoria	ACT-27	Redacción y estructuración técnica de la memoria en LaTeX
F-10 – Defensa	ACT-28	Diseño, desarrollo y maquetación de diapositivas de defensa
	ACT-29	Ensayos de oratoria y preparación de la defensa pública

Tabla 3.24: Actividades organizadas por su fase del cronograma.

3.3.3. Lista de hitos

Los hitos representan los puntos de control clave y logros de duración cero dentro del cronograma del proyecto. Su consecución suponen grandes avances en el desarrollo o la finalización de fases completas, y permiten evaluar el progreso general del proyecto.

Fase	ID	Hito de logro	Límite
F-01 – Planificación	H-01	Acta de constitución del proyecto redactada y firmada	21 Ene
	H-02	Líneas base del proyecto constituidas y aprobadas	28 Ene
F-02 – Documentación	H-03	Catálogo de requisitos, casos de uso y modelos de datos definidos	18 Feb
	H-04	Manuales técnicos de usuario y despliegue finalizados	14 Abr
F-03 – Inicio	H-05	Repositorio inicializado y entorno de desarrollo listo	11 Feb
F-04 – CI/CD	H-06	Flujo de CI/CD automatizado y plantillas de trabajo desplegadas	1 Mar
F-05 – MVP	H-07	Funcionalidades del MVP operativas (creación y respuesta de tests)	8 Mar
F-06 – Testing	H-08	Baterías de pruebas de software ejecutadas y superadas con éxito	26 Abr
F-07 – Core	H-09	Conexión y respuesta a cuestionarios en tiempo real operativos	29 Mar
	H-10	Motor de validación y control de integridad de respuestas completado	12 Abr
F-08 – QA	H-11	Vistas de historial de salas pasadas y exportación de datos funcionales	19 Abr
F-09 – Memoria	H-12	Memoria académica consolidada y entregada	17 May
F-10 – Defensa	H-13	Presentación y material de soporte maquetados para la defensa	5 Jun
	H-14	Defensa pública del proyecto ejecutada ante el tribunal	10 Jun

Tabla 3.25: Hitos de control del proyecto.

3.3.4. Diagrama de Gantt

El siguiente diagrama de Gantt sintetiza de manera visual la distribución cronológica del proyecto, clave para el seguimiento del trabajo realizado y asegurar el cumplimiento de los plazos fijados.

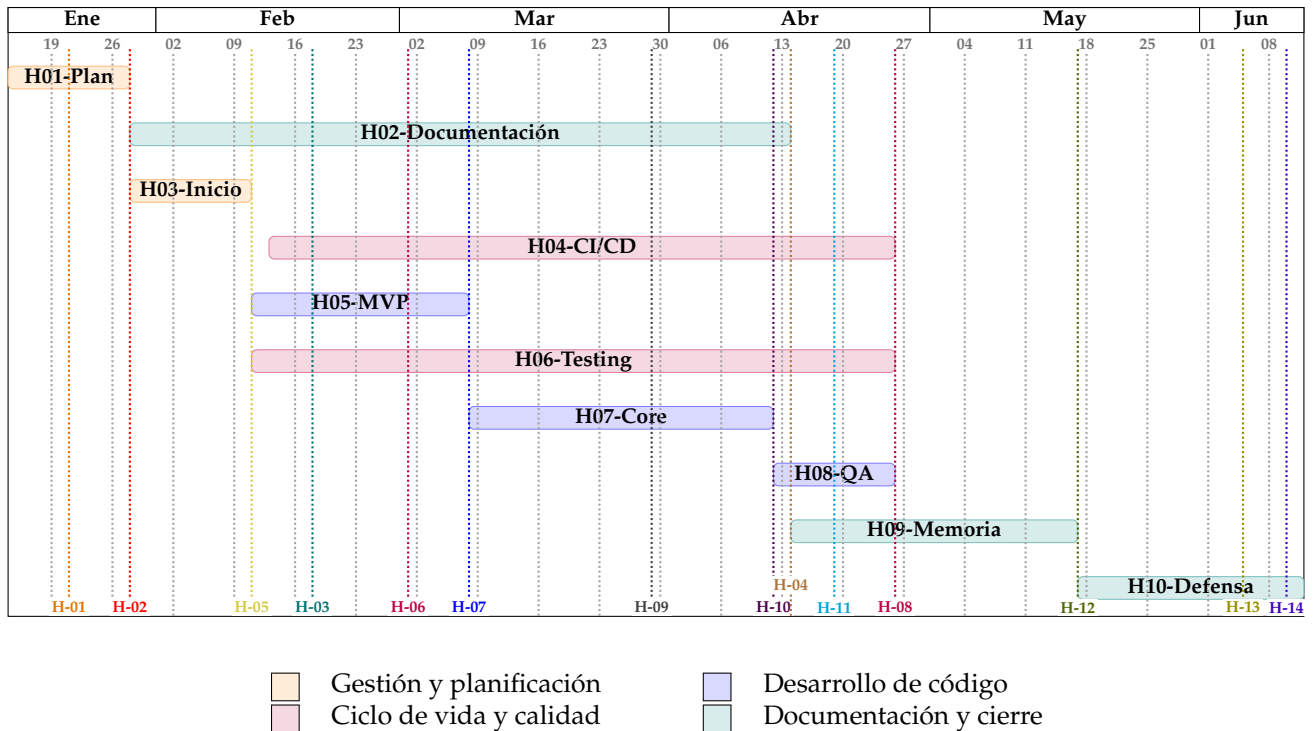


Figura 3.2: Diagrama de Gantt inicial del proyecto.

3.4. Línea base de Costes

El desarrollo de un proyecto como *Quizzie* no solo exige viabilidad técnica, sino también un modelo financiero sostenible que justifique su implantación frente a otras soluciones. Para ello, se analizan a continuación los costes operativos, el impacto económico que supone el escalado de la infraestructura y el Coste Total de Propiedad (TCO).

3.4.1. Análisis de costes

El estudio económico de la plataforma se fundamenta en dos componentes financieros esenciales: los Gastos de Capital (*Capital Expenditure* o CAPEX), que engloban el coste teórico de desarrollo y los recursos fijos necesarios para materializar el sistema, y los Gastos Operativos (*Operational Expenditure* o OPEX), que representan el coste mensual estimado para la explotación de la plataforma en producción.

Concepto	Coste neto
Costes laborales	10.865,49 €
Equipamiento informático	500,00 €
Licencia comercial anual Google AI Premium (Gemini Pro)	218,07 €
Licencia GitHub Team (6 meses)	21,82 €
Licencia Microsoft Teams Essentials (6 meses)	21,82 €
Reserva de contingencia	1.162,72 €
Total CAPEX	12.789,92 €

Tabla 3.26: Resumen de Gastos de Capital (CAPEX).

Para justificar coste laboral del CAPEX, se adopta un modelo de roles mixtos debido a que el proyecto ha sido ejecutado íntegramente por un único ingeniero. De este modo, cada actividad de la EDT se presupuesta teniendo en cuenta el rol profesional que la llevará a cabo. Para garantizar el rigor y la objetividad de las tarifas aplicadas, los precios base por hora se han extraído del *Informe Final sobre la Consulta Preliminar del Mercado: “Perfiles Profesionales Ámbito Informático”* de la Junta de Andalucía [1]. Siguiendo las directrices de dicho organismo, se utiliza como referencia la métrica de la **media acotada**, un indicador estadístico corregido mediante el test de Tukey que elimina valores atípicos del sector. Sobre la base salarial neta resultante de 8.358,07 €, se ha aplicado un recargo del 30% en concepto de Seguridad Social y costes empresariales de contratación directos del mercado español, consolidando el coste laboral real en **10.865,49 €**. El desglose detallado de las horas por actividad y perfiles profesionales que sustenta este cálculo se presenta en la tabla de [costes netos laborales](#).

Recurso (Rol)	Actividad	Precio/hora	Horas	Coste neto
Project Manager	ACT-01	46,09 €	4 h	184,36 €
	ACT-02	46,09 €	4 h	184,36 €
	ACT-14	46,09 €	5 h	230,45 €
	ACT-15	46,09 €	3 h	138,27 €
	ACT-29	46,09 €	10 h	460,90 €
	Subtotal		26 h	1.198,34 €
Analista	ACT-06	38,33 €	2 h	76,66 €
	ACT-08	38,33 €	3 h	114,99 €
	ACT-09	38,33 €	4 h	153,32 €
	ACT-10	38,33 €	4 h	153,32 €
	ACT-11	38,33 €	4 h	153,32 €
	ACT-27	38,33 €	70 h	2.683,10 €
	Subtotal		87 h	3.334,71 €
Diseñador UI	ACT-03	33,43 €	2 h	66,86 €
	ACT-04	33,43 €	5 h	167,15 €
	ACT-05	33,43 €	5 h	167,15 €
	ACT-28	33,43 €	10 h	334,30 €
	Subtotal		22 h	735,46 €
Programador Backend	ACT-12	23,85 €	2 h	47,70 €
	ACT-16	23,85 €	25 h	596,25 €
	ACT-18	23,85 €	20 h	477,00 €
	ACT-21	23,85 €	20 h	477,00 €
	ACT-23	23,85 €	12 h	286,20 €
	ACT-24	23,85 €	5 h	119,25 €
	Subtotal		84 h	2.003,40 €
Programador Frontend	ACT-17	23,85 €	35 h	834,75 €
	ACT-22	23,85 €	20 h	477,00 €
	ACT-25	23,85 €	5 h	119,25 €
	Subtotal		60 h	1.431,00 €
Tester	ACT-19	23,68 €	7 h	165,76 €
	ACT-20	23,68 €	8 h	189,44 €
	Subtotal		15 h	355,20 €
Técnico	ACT-07	19,58 €	1 h	19,58 €
	ACT-13	19,58 €	2 h	39,16 €
	ACT-26	19,58 €	3 h	58,74 €
	Subtotal		6 h	117,48 €
TOTAL			300 h	8.358,07 €

Tabla 3.27: Costes netos laborales agrupados por perfil profesional.

Concepto	Coste neto
Infraestructura web Frontend (<i>Vercel Pro</i>)	18,18 €/mes
Servidor de aplicaciones y Base de Datos (<i>Render Starter</i>)	12,73 €/mes
Consumo de la API de <i>Gemini Pro</i> para el asistente de IA	15,00 €/mes
Total OPEX	45,91 €/mes

Tabla 3.28: Resumen de Gastos Operativos mensuales (OPEX).

Cabe destacar que los Gastos Operativos (OPEX) reflejados en la [tabla](#) no constituyen un coste estático, sino que representan una proyección basada en una estimación inicial de 500 usuarios activos mensuales, asumiendo un consumo moderado de recursos y una utilización estándar de la API de inteligencia artificial. En escenarios de escalado masivo, donde el volumen de usuarios concurrentes o la demanda de cómputo aumente significativamente, es previsible que estos costes operativos se incrementen de forma proporcional. Por ello, en la siguiente subsección se detalla y aclara matemáticamente el comportamiento financiero de la plataforma ante diferentes escenarios de carga y volumen de usuarios.

3.4.2. Costes de infraestructura según el número de usuarios

Para prever el impacto financiero de la plataforma en entornos reales, se analiza el comportamiento de los costes operativos (OPEX) bajo tres escenarios de demanda: pesimista, conservador y optimista. Esta clasificación permite evaluar la elasticidad de la infraestructura en la nube (*Render* y *Vercel*) y el consumo variable de la API de *Gemini* según el volumen de uso.

Concepto	Pesimista	Conservador	Optimista
Infraestructura web Frontend (<i>Vercel Pro</i>)	0,00 €	18,18 €	18,18 €
Servidor de aplicaciones y Base de Datos (<i>Render Starter</i>)	0,00 €	12,73 €	27,27 €
Consumo de la API de <i>Gemini Pro</i> para el asistente de IA	2,50 €	15,00 €	60,00 €
Total OPEX	2,50 €/mes	45,91 €/mes	105,45 €/mes

Tabla 3.29: Comparativa de OPEX según escenario de escalado.

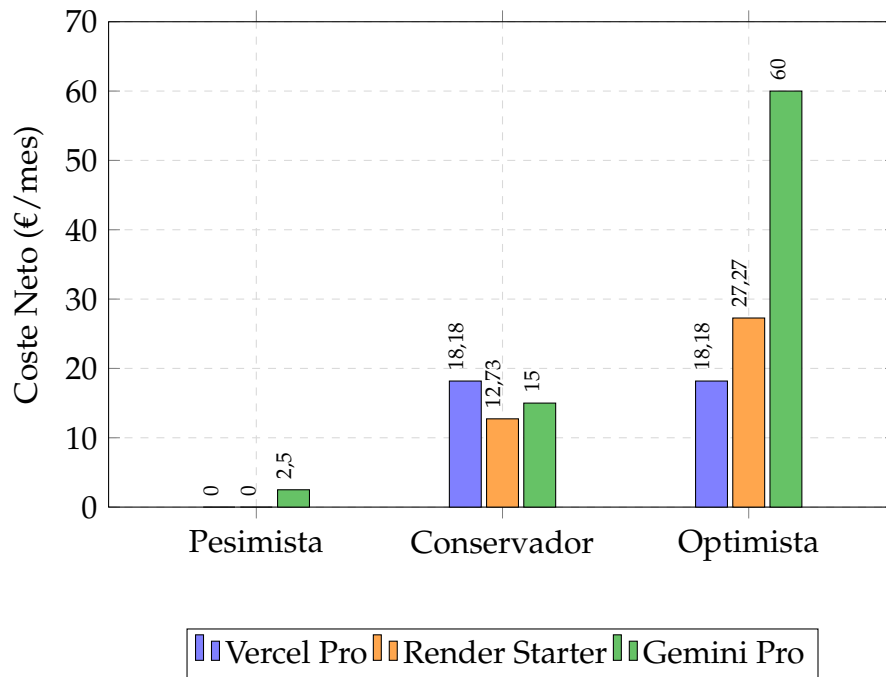


Figura 3.3: Costes mensuales de infraestructura según escenario de escalado.

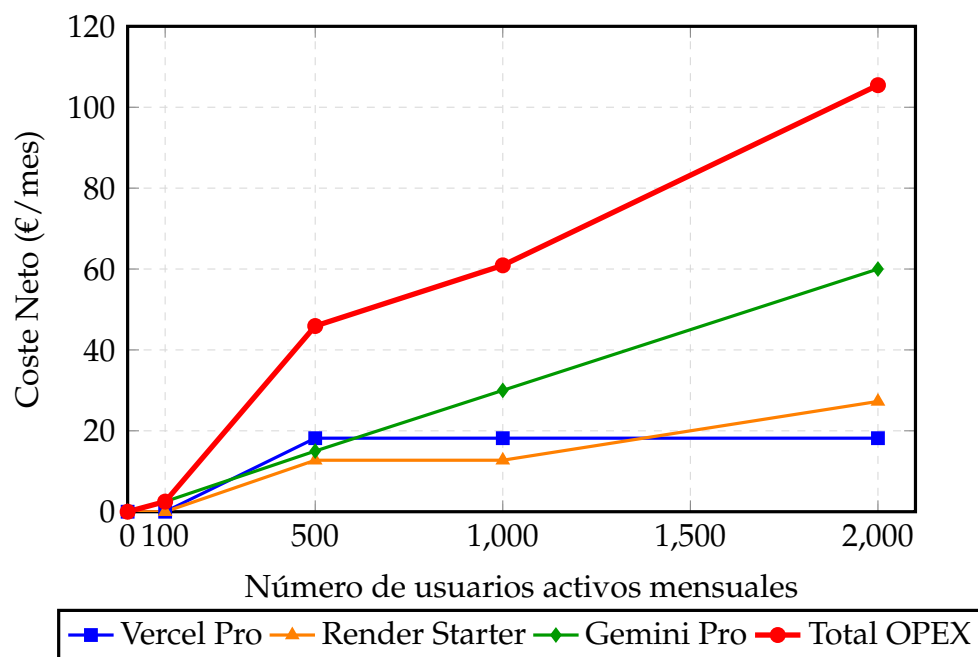


Figura 3.4: Evolución de OPEX en función del volumen de usuarios.

3.4.3. Coste total de propiedad (TCO)

El costo total de propiedad (*Total Cost of Ownership*, TCO) permite evaluar el impacto económico global de la plataforma a medio plazo al consolidar la inversión inicial de desarrollo (CAPEX) junto con los gastos operativos (OPEX) proyectados a tres años. Para garantizar la validez metodológica de este presupuesto frente al mercado real, los costes de la mano de obra se contrastan con el *Informe final sobre la consulta preliminar del mercado* de la Junta de Andalucía [2]. De este modo, asumiendo los gastos de infraestructura del escenario conservador, la proyección resultante se detalla en la siguiente tabla.

Concepto	Año 1	Año 2	Año 3
Inversión inicial (CAPEX)			
Desarrollo de Software (Ingeniería)	5.935,20 €	0,00 €	0,00 €
Gastos operativos (OPEX)			
Infraestructura Frontend (<i>Vercel Pro</i>)	218,16 €	218,16 €	218,16 €
Servidor de aplicaciones y BD (<i>Render Starter</i>)	152,76 €	152,76 €	152,76 €
Consumo de la API de <i>Gemini Pro</i>	180,00 €	180,00 €	180,00 €
Total anual	6.486,12 €	550,92 €	550,92 €
TCO acumulado (3 Años)	7.587,96 €		

Tabla 3.30: TCO proyectado a tres años.

Nota: No se han imputado costes de personal ni horas de mantenimiento dentro del OPEX debido a que el esfuerzo laboral se ha contabilizado íntegramente como parte del CAPEX, contando con las 300 horas reglamentarias del TFG.

3.5. Plan de gestión de cambios y riesgos

Un desarrollo tecnológico está inherentemente sujeto a imprevistos y desviaciones. Para asegurar el éxito y la entrega en plazo de la plataforma, este plan de contingencia se apoyará de forma sistemática en la matriz de riesgos iniciales definida en la [Sección 2.5](#). En caso de que alguna de las amenazas identificadas se materialice, o bien si surge la necesidad de aplicar cualquier variación sobre los requisitos o la planificación del sistema, se activará el procedimiento formal de control estructurado a continuación.

Cualquier propuesta de modificación se gestionará siguiendo este flujo secuencial:

1. **Identificación:** Toda propuesta de modificación se registrará en el *backlog*, especificando si afecta al código de la aplicación, a la redacción de la memoria o a la planificación temporal del proyecto.

2. **Análisis de impacto:** Evaluar el esfuerzo y las horas necesarias para aplicar el cambio, estimando cómo afectará al calendario global de las asignaturas, a la documentación pendiente o a la arquitectura del sistema.
3. **Resolución:** Decidir de forma justificada si el cambio se aprueba para su ejecución inmediata, se pospone como una mejora futura o se descarta si compromete críticamente los plazos de entrega del TFG.
4. **Implementación y despliegue:** Se ejecuta el cambio en el área correspondiente (programación de código, reajuste de la planificación o corrección de la memoria), realizando las pruebas o revisiones pertinentes antes de dar por actualizada la documentación del proyecto.

4. Estado del arte y fundamentación

El éxito de una plataforma tecnológica no solo radica en su correcto funcionamiento, sino en su capacidad para aportar valor real frente a las soluciones existentes y sustentarse sobre decisiones de ingeniería justificadas. Antes de iniciar el diseño arquitectónico, es clave analizar el ecosistema competitivo y evaluar críticamente las herramientas de desarrollo disponibles en la industria actual.

A lo largo de las siguientes secciones se realiza un análisis del mercado de plataformas de evaluación mediante cuestionarios para identificar sus limitaciones y fijar los factores diferenciadores de *Quizzie*. Asimismo, se detalla la fundamentación tecnológica del proyecto, examinando las alternativas consideradas y justificando la selección del stack definitivo, para concluir con la síntesis estratégica del estudio previo.

4.1. Análisis del mercado

Para fundamentar la necesidad de desarrollar una nueva plataforma de evaluación, es imprescindible analizar el ecosistema de soluciones empleadas actualmente en el ámbito educativo. Aunque el mercado cuenta con soluciones consolidadas, el estudio de sus características revela limitaciones críticas en el control de la presencialidad, la exportación de datos y la asistencia al profesorado. A continuación, se examinan las tres alternativas más representativas:

- **Kahoot!:** Pionera en el ámbito de la gamificación educativa, destaca por su alta capacidad de fidelización y su interfaz visual atractiva. Sin embargo, su enfoque lúdico penaliza la rigurosidad académica: prioriza la velocidad de respuesta sobre la reflexión profunda, apenas ofrece analíticas avanzadas exportables en sus versiones gratuitas y carece de cualquier mecanismo que impida a un alumno participar de forma remota compartiendo el código PIN de la sala.
- **Wooclap:** Orientada a un entorno universitario y corporativo más formal, ofrece una excelente variedad de tipos de preguntas y herramientas de interacción en vivo. Su principal barrera reside en la usabilidad y la agilidad de los flujos de trabajo; el proceso de diseño de cuestionarios extensos resulta monótono y manual para el docente, y no tiene funcionalidades inteligentes de automatización que agilicen la generación de contenido académico.
- **Cuestionarios de Enseñanza Virtual:** Las plataformas institucionales integradas ofrecen una robustez indiscutible a nivel de seguridad, evaluación formal y exportación nativa de calificaciones. No obstante, sacrifican por completo la inmediatez y la experiencia de usuario. Su interfaz rígida, la complejidad técnica para configurar sesiones en tiempo real y la

imposibilidad de mitigar de forma sencilla el fraude por suplantación digital a distancia limitan su efectividad como dinámicas de aula ágiles.

Frente a este escenario, se ha sintetizado una comparativa directa en la [Tabla 4.1](#), evaluando las características esenciales para la docencia presencial moderna. Las carencias detectadas en las alternativas comerciales e institucionales delimitan el espacio de oportunidad para una solución integral.

Característica	Kahoot!	Wooclap	Enseñanza Virtual	Quizzie
Evaluación en tiempo real	Sí	Sí	Limitado	Sí
Acceso rápido (sin registro de alumnos)	Sí	Sí	No	Sí
Verificación de presencialidad física	No	No	No	Sí
Generación de test asistida por IA	No	Limitado	No	Sí
Creación ágil de cuestionarios	Sí	Limitado	No	Sí
Visualización de estadísticas y analíticas	Sí	Sí	Limitado	Sí
Exportación de resultados	De pago	Limitado	Sí	Sí

Tabla 4.1: Tabla comparativa del mercado.

4.2. Factores diferenciadores

Como respuesta a las debilidades del mercado identificadas en la sección anterior, *Quizzie* se diseña no como una alternativa lúdica adicional, sino como un entorno equilibrado que fusiona el dinamismo interactivo con la formalidad académica. Sus elementos innovadores y ventajas competitivas se estructuran en los siguientes pilares:

- **Validación de presencia física mediante QR:** Para evitar el fraude por suplantación o que los alumnos participen a distancia, la plataforma genera un código QR único en el dispositivo del estudiante justo al finalizar el test, el cual incluye su nombre y la nota obtenida. El docente es quien escanea este código desde su propio dispositivo al terminar la clase, confirmando de manera inequívoca la asistencia física del alumno y registrando su calificación en el acto.
- **Creación inteligente asistida por IA:** Para resolver el flujo de trabajo monótono de diseñar cuestionarios desde cero, *Quizzie* integra un motor de Inteligencia Artificial que asiste al docente generando automáticamente baterías de preguntas coherentes a partir del temario o texto introducido, agilizando drásticamente la preparación de la clase.
- **Exportación directa de calificaciones:** La plataforma simplifica la gestión de las calificaciones permitiendo descargar las notas de los alumnos directamente en formatos estándar como CSV o PDF, listos para su revisión o publicación en canales oficiales.

- **Accesibilidad y agilidad en el aula:** Los alumnos acceden de forma inmediata sin necesidad de registros previos ni cuentas obligatorias, mientras que el profesor dispone de un panel simplificado para lanzar y controlar el flujo del cuestionario con un solo clic.

4.3. Fundamentación tecnológica

A continuación, se presenta la arquitectura tecnológica que da soporte a *Quizzie*. Como vista general de la infraestructura, la siguiente tabla muestra la distribución definitiva de las herramientas seleccionadas siguiendo un orden ascendente desde la persistencia hasta el despliegue del cliente.

Base de datos	SQLite
ORM / Validación	SQLModel
Backend	FastAPI (Python)
Lenguaje frontend	TypeScript
Frontend (SPA)	React + Vite
Estilos frontend	CSS <i>vanilla</i>
Alojamiento servidor	Render
Alojamiento cliente	Vercel
Flujo de CI/CD	GitHub Actions

Tabla 4.2: Resumen del stack tecnológico de Quizzie.

4.3.1. Alternativas consideradas y justificación de las decisiones

Para fundamentar la elección de cada componente del sistema, se analizan a continuación las matrices comparativas correspondientes a cada uno de los bloques lógicos de la aplicación.

Persistencia de datos

La gestión del almacenamiento del proyecto requiere evaluar la estructura de los datos frente a la complejidad de la infraestructura en la nube.

Tecnología	Ventajas	Desventajas
Opción elegida		
SQLite	Portabilidad en un solo archivo; cero administración de red y coste económico nulo.	Bloqueo del archivo completo durante la concurrencia de escritura masiva.
Alternativas valoradas		
PostgreSQL	Escalabilidad masiva y gestión avanzada de accesos y concurrencia.	Elevada complejidad de despliegue y administración de accesos en producción.
MySQL	Motor relacional maduro, robusto y con gran soporte de la comunidad.	Costes económicos asociados a instancias en la nube innecesarios en esta fase.
MongoDB	Flexibilidad de esquemas basada en documentos NoSQL y alta velocidad de lectura.	Descartado porque el dominio de datos del proyecto es puramente relacional.

Tabla 4.3: Alternativas tecnológicas para la base de datos.

Mapeo objeto-relacional (ORM)

Se evalúa la forma de interactuar con el motor relacional buscando la mayor limpieza y eficiencia en el código.

Tecnología	Ventajas	Desventajas
Opción elegida		
SQLModel	Cumple el principio DRY: fusiona eficientemente esquemas de API y modelos de BD.	Herramienta más reciente en el ecosistema, con menor volumen de hilos de soporte.
Alternativas valoradas		
SQLAlchemy + Pydantic	Estándar clásico de la industria con máxima madurez y documentación.	Duplicidad de código obligatoria al tener que declarar los modelos por separado.

Tabla 4.4: Alternativas tecnológicas para el ORM y validación.

Backend y servidor de sockets

La lógica de negocio exige un entorno que soporte una alta concurrencia interactiva en tiempo real.

Tecnología	Ventajas	Desventajas
Opción elegida		
FastAPI	Rendimiento asíncrono nativo (<i>async/await</i>) ideal para sockets.	Estructura menos guiada en comparación con frameworks monolíticos.
Alternativas valoradas		
Node.js (Express / NestJS)	Estándar tradicional para WebSockets; unifica el lenguaje del proyecto a JavaScript.	Se descartó para priorizar el uso de Python debido a su ecosistema de IA.
Django	Framework muy robusto, maduro y con arquitectura <i>batteries-included</i> .	Estructura fundamentalmente síncrona por defecto y peso excesivo para una API REST.
Flask	Microframework extremadamente simple, minimalista y rápido de configurar.	Carece de herramientas nativas de validación y de soporte asíncrono integrado.

Tabla 4.5: Alternativas tecnológicas para el desarrollo del backend.

Lenguaje del cliente web

Se contrasta la seguridad del tipado frente a la velocidad de desarrollo directa en el frontend.

Tecnología	Ventajas	Desventajas
Opción elegida		
TypeScript	Tipado estático estricto; mitiga errores con contratos claros para WebSockets.	Requiere tiempo adicional en el ciclo de desarrollo para configurar interfaces.
Alternativas valoradas		
JavaScript <i>vanilla</i>	Curva de aprendizaje nula y compatibilidad nativa directa en el navegador.	Falta de seguridad de tipos en la mensajería asíncrona en tiempo real.

Tabla 4.6: Alternativas para el lenguaje de programación frontend.

Frontend (interfaz de usuario)

Se analiza la arquitectura de las interfaces y la velocidad en la compilación del entorno de desarrollo.

Tecnología	Ventajas	Desventajas
Opción elegida		
React + Vite	<i>Virtual DOM</i> reactivo, ecosistema masivo y ciclos de compilación HMR instantáneos.	Curva de aprendizaje inicial superior frente a opciones más básicas.
Alternativas valoradas		
Angular	Framework corporativo muy sólido, estructurado y altamente escalable.	Curva de aprendizaje muy empinada y verbosidad excesiva para el alcance del TFG.
Vue.js	Alternativa equilibrada con un sistema de reactividad sencillo y progresivo.	Ecosistema y penetración en el mercado laboral inferior en comparación con React.
Svelte	Máxima ligereza al eliminar el <i>virtual DOM</i> y compilar a código nativo.	Comunidad reducida y menor disponibilidad de librerías de terceros maduras.
Webpack	Estándar histórico para el empaquetado de aplicaciones web.	Procesos de construcción muy lentos en desarrollo local comparado con Vite.

Tabla 4.7: Alternativas tecnológicas para la interfaz de usuario.

Estilización visual de la interfaz

Se evalúa el equilibrio entre la velocidad de diseño y la sobrecarga estética en la aplicación del cliente.

Tecnología	Ventajas	Desventajas
Opción elegida		
CSS <i>vanilla</i>	Control total de layouts y animaciones sin dependencias ni sobrecarga.	Requiere la escritura manual de todos los estilos desde cero.
Alternativas valoradas		
Tailwind CSS	Maquetación veloz mediante clases de utilidad atómicas directas.	Saturación excesiva del marcado JSX, perjudicando la legibilidad del código.
Bootstrap	Componentes preconstruidos adaptables y fáciles de implementar.	Estética genérica y obsoleta; complejiza la personalización visual premium.

Tabla 4.8: Alternativas tecnológicas para el diseño de estilos visuales.

Infraestructura y despliegue

Se analizan los entornos de alojamiento priorizando la compatibilidad con canales bidireccionales y la viabilidad económica.

Tecnología	Ventajas	Desventajas
Opción elegida		
Vercel + Render	Plan gratuito sin tarjeta, GitOps automatizado y soporte ASGI para WebSockets.	El plan gratuito de Render sufre de retardos iniciales por <i>cold start</i> .
Alternativas valoradas		
AWS	Control absoluto e ilimitado de la infraestructura de red.	Configuración excesivamente compleja y riesgo de costes económicos imprevistos.
Heroku	Entorno PaaS históricamente óptimo para servidores de sockets interactivos.	Definición obsoleta por la eliminación por completo de sus planes gratuitos.

Tabla 4.9: Alternativas para la infraestructura y el despliegue.

4.3.2. Limitaciones tecnológicas

A pesar de la idoneidad del stack seleccionado, se han identificado las siguientes limitaciones técnicas inherentes a las decisiones tomadas:

- **Cold start en Render (plan gratuito):** El contenedor del backend se desactiva tras 15 minutos sin recibir peticiones. Esto genera un retardo inicial de entre 30 y 50 segundos en la primera interacción de un usuario mientras el contenedor vuelve a iniciarse.
- **Concurrencia de escritura en SQLite:** SQLite bloquea la base de datos a nivel de archivo completo durante las operaciones de escritura. Aunque es imperceptible para un grupo estándar de alumnos, limitaría el escalado para miles de usuarios simultáneos registrando respuestas al unísono.
- **Falta de SSR (*server-side rendering*) en el frontend:** Al ser una SPA de React pura, el indexado por motores de búsqueda (SEO) es limitado. Sin embargo, al tratarse de una aplicación académica de acceso cerrado mediante credenciales, esta limitación no compromete su viabilidad.

4.3.3. Herramientas de soporte al desarrollo y control de calidad

La robustez del sistema y el cumplimiento de los estándares de calidad se articulan mediante las siguientes herramientas de soporte:

- **Control de versiones y gestión del proyecto:** Se utiliza **Git** como sistema de control de versiones local y **GitHub** como plataforma de alojamiento remoto. Para la organización y planificación de las tareas del cronograma, se emplea **GitHub Projects** estructurado como un tablero *Kanban*, asegurando la trazabilidad del desarrollo mediante la vinculación de *issues* basadas en

plantillas personalizadas (*issue templates*) para la apertura de errores o nuevas características.

- **Estandarización de commits:** Para garantizar un historial de Git limpio, semántico y legible, se integra **Commitizen**. Esta herramienta obliga al autor a seguir la convención de *Conventional Commits*, categorizando de forma estricta cada cambio (por ejemplo, *feat*, *fix*, *docs* o *refactor*), lo que facilita la auditoría del código y la futura generación automatizada de diarios de cambios (*changelogs*).
- **Entorno de desarrollo integrado (IDE):** El entorno principal de trabajo es **Visual Studio Code**, seleccionado por su ligereza y su ecosistema de extensiones. Entre ellas, se ha configurado **LaTeX Workshop** para centralizar la composición y compilación local de la memoria académica mediante los motores de **MiKTeX** y **Strawberry Perl**.
- **Gestión de entornos y dependencias:** En el entorno de backend se ha adoptado **uv**, una herramienta de gestión de paquetes y entornos virtuales para Python extremadamente rápida escrita en Rust, que sustituye de forma eficiente a herramientas tradicionales como *pip* y *poetry*. Para el entorno frontend se utiliza **npm** como gestor de paquetes estándar de Node.js.
- **Generación y lectura de códigos QR:** Para dar soporte al sistema de validación de presencia física, en el frontend se integra la librería **qrcode.react** para renderizar de manera eficiente y reactiva el código QR dinámico en la pantalla del alumno al finalizar el test. Por la parte del docente, se utiliza **html5-qrcode** para acceder a la cámara del dispositivo móvil y procesar el escaneo en tiempo real con baja latencia.
- **Linter y formateo:** En el backend se utiliza **Ruff**, *linter* y formateador ultrarrápido programado en Rust que optimiza el cumplimiento de PEP 8 y unifica tareas que antes requerían múltiples librerías independientes (como *Flake8*, *Black* e *isort*). En el frontend se emplea **ESLint** con *flat config* para controlar la calidad de TypeScript junto a **Prettier** para garantizar un formateo de código unificado y consistente.
- **Pre-commit hooks:** Con la librería **pre-commit**, se interceptan los *commits* locales para garantizar que solo se sube código limpio que supere de forma automatizada las reglas de formato, tipado y análisis estático antes de salir del entorno local.
- **Pruebas unitarias y de integración:** Se utiliza **pytest** en el backend junto a **pytest-cov** para el cálculo de cobertura, y **Vitest** con **React Testing Library** en el frontend para evaluar de manera aislada el comportamiento de los componentes. Para flujos de usuario extremo a extremo (E2E), se integra **Playwright**, permitiendo simular múltiples navegadores concurrentes.
- **Pruebas de carga:** Se emplea **Locust** para verificar de manera empírica la capacidad del servidor y medir la latencia del protocolo WebSocket al soportar la avalancha de respuestas concurrentes de los alumnos.
- **SonarCloud:** Se ha configurado este auditor externo de análisis estático como

parte del flujo de CI/CD en GitHub Actions, exigiendo una meta de calidad estricta (*quality gate* A, duplicación $< 3\%$, cobertura de tests $> 80\%$).

4.4. Conclusiones del estudio previo

Decir por qué veo necesario el proyecto, qué aporta y cómo se va a llevar a cabo

5. Análisis de requisitos

Aquí se describirán los objetivos del sistema

5.1. Actores del sistema

Alumnos, profesores (si procede, también administrador)

5.2. Historias de usuario

Como [actor], quiero [acción] para [beneficio]”.

Valorar si incluirlos todos en una diagrama estilo draw.io

5.3. Requisitos de información

5.4. Reglas de negocio y restricciones

5.5. Requisitos funcionales

5.5.1. Criterios de aceptación

Se puede poner dentro de los RF como subsección o fuera como sección, enlazar con los casos de prueba y validación (cap 8.3)

5.6. Casos de uso

Descripción y después diagrama parecido al del de historias de usuario, pero un diagrama por CU

5.7. Requisitos no funcionales

Requisitos de rendimiento, seguridad, usabilidad, etc.

5.8. Trazabilidad de requisitos

Matriz de trazabilidad : Objetivos $\rightarrow$ Requisitos $\rightarrow$ Diseño $\rightarrow$ Pruebas $\rightarrow$ Resultados

6. Diseño y arquitectura

6.1. Patrón arquitectónico elegido

Arquitectura Cliente-Servidor, arquitectura en capas y arquitectura basada en eventos

Hablar de los patrones arquitectónicos elegidos, explicando por qué se han seleccionado y cómo se aplican en el diseño de la aplicación, destacando la separación de responsabilidades y la escalabilidad que ofrecen.

6.2. Estructura modular del código

Organización del proyecto en módulos independientes tanto para el Frontend (React) como para el Backend (FastAPI), facilitando la escalabilidad y el mantenimiento

Hablar de la organización del código en módulos independientes, explicando cómo se han estructurado y por qué es importante para la escalabilidad y el mantenimiento del proyecto.

6.3. Modelo de datos

Diagrama de dominio

Incluir el código mermaid y el diagrama

6.4. Diseño de interfaces

Mockups y resultado final

Mostrar los mockups iniciales y compararlos con el resultado final, explicando las decisiones de diseño tomadas y cómo se han implementado en la interfaz de usuario.

6.5. Seguridad

Autenticación JWT, gestión de roles y tokens de verificación para participantes

6.6. Documentación de la API

Descripción de los endpoints principales de la API, detallando las peticiones, respuestas y la integración con el estándar OpenAPI

Aquí exprimiría el Swagger

7. Metodología técnica (CI/CD)

Mi idea es haber explicado todas las herramientas en el capítulo 4, pero aquí se podría entrar más en detalle sobre cómo se han configurado y utilizado en lo que respecta a ci/cd, gestión de ramas, commits, etc. También se podrían incluir políticas de código, como pre-commits, hooks, plantillas de issues, etc. que no se hayan mencionado en capítulos anteriores.

7.1. Políticas del repositorio

Ramas, commits y versiones. Se puede mencionar los pre-commits, hooks, Github, Commitizen, Convenciones de commits, plantillas de issues, ... y su importancia en el desarrollo para trabajar de manera más organizada

7.2. Integración y Despliegue Continuo (CI/CD)

Descripción del ciclo ci/cd

7.2.1. Configuración del Pipeline y automatización

Implementación de flujos de trabajo en GitHub Actions y qué herramientas incluyen

7.3. Perfiles de configuración y entornos

Variables de entorno, secretos y configuración específica para los entornos de desarrollo, pruebas y producción.

7.4. Métricas de calidad y análisis estático de código

SonarCloud para el análisis estático de código, deuda técnica y cobertura de pruebas. (Para ello también habría que ver hasta qué punto se ha hablado de Sonar en capítulos anteriores)

Aquí quiero hacer especial énfasis para explicar la importancia de estas métricas, cómo se han configurado y qué resultados han dado, mostrando gráficas o ejemplos concretos de cómo han ayudado a mejorar la calidad del código y a

identificar áreas de mejora. También es importante recalcar que su uso es vital para evitar deuda técnica, y en el capítulo 9 de resultados diré que me arrepiento de haberlo implementado tan tarde y que me ha hecho tener que aplazar un par de issues que tenias planteadas para la entrega del tfg. (también podría ser en el capítulo de conclusiones)

8. Implementación y pruebas

8.1. Componentes clave

Fragmentos de código fundamentales

8.2. Estrategia de testing

Unitarias, integración, E2E, etc

Me gustaría mencionar también su importancia y los objetivos de cobertura que tengo, quizás lo mejor sea en este capítulo mencionar los umbrales objetivos y en el capítulo 9 de resultados hablar de su cumplimiento

8.3. Casos de prueba y validación

Descripción de los casos de prueba utilizados para validar los criterios de aceptación y alcanzar la cobertura objetivo

8.4. Pruebas de carga y rendimiento

Descripción de las pruebas de carga y rendimiento realizadas

Importante porque los cuestionarios son en directo y con muchas personas simultáneamente

9. Resultados y evaluación

9.1. Grado de cumplimiento de la Línea Base del Alcance

Evaluación del cumplimiento de los objetivos y requisitos establecidos en la Línea Base del Alcance.

9.2. Tiempo real frente a Línea Base del Cronograma

Comparación del tiempo real dedicado a cada fase del proyecto con el tiempo estimado inicialmente

9.3. Gastos incurridos frente a Línea Base de Costes

Diferencia entre los gastos previstos y los reales

9.4. Desviaciones y gestión de cambios realizados

Cómo se gestionaron las desviaciones y qué modificaciones de la planificación provocaron

9.5. Resultados de las pruebas y validación

Resultados obtenidos en las pruebas unitarias, de integración, E2E, carga y rendimiento, y su comparación con los objetivos establecidos

9.6. Retrospectiva técnica

Reflexión sobre la implementación de las prácticas de CI/CD, testing y métricas de calidad, y decir si he cumplido los objetivos como las métricas de cobertura o de issues de sonar

10. Conclusiones

10.1. Grado de cumplimiento de los objetivos

Evaluación sobre los objetivos planteados en el acta de constitución.

10.2. Lecciones aprendidas y valoración personal

Reflexión sobre el trabajo realizado.

10.3. Trabajos futuros y líneas de evolución

Conclusiones sobre el trabajo realizado y líneas de evolución futuras.

Manual de usuario

Manual de usuario de quizzie

Manual de instalación

Guía de instalación

Bibliografía

- [1] Junta de Andalucía. Informe final sobre la consulta preliminar del mercado: “perfiles profesionales Ámbito informático”. Informe técnico, Consejería de Agricultura, Pesca y Desarrollo Rural. Secretaría General Técnica. Servicio de Informática, Sevilla, España, 2018. Firmado por D. Juan Luis Ceada Ramos. Publicado en el Perfil del Contratante.

- [2] Junta de Andalucía. Informe final sobre la consulta preliminar del mercado: “perfiles profesionales en el ámbito de consultoría de compra pública de innovación”. Informe técnico, Consejería de Agricultura, Ganadería, Pesca y Desarrollo Sostenible, Sevilla, España, 2019. URL <https://juntadeandalucia.es/temas/contratacion-publica/perfiles-licitaciones/consultas-preliminares/detalle/000000169185.html>.